

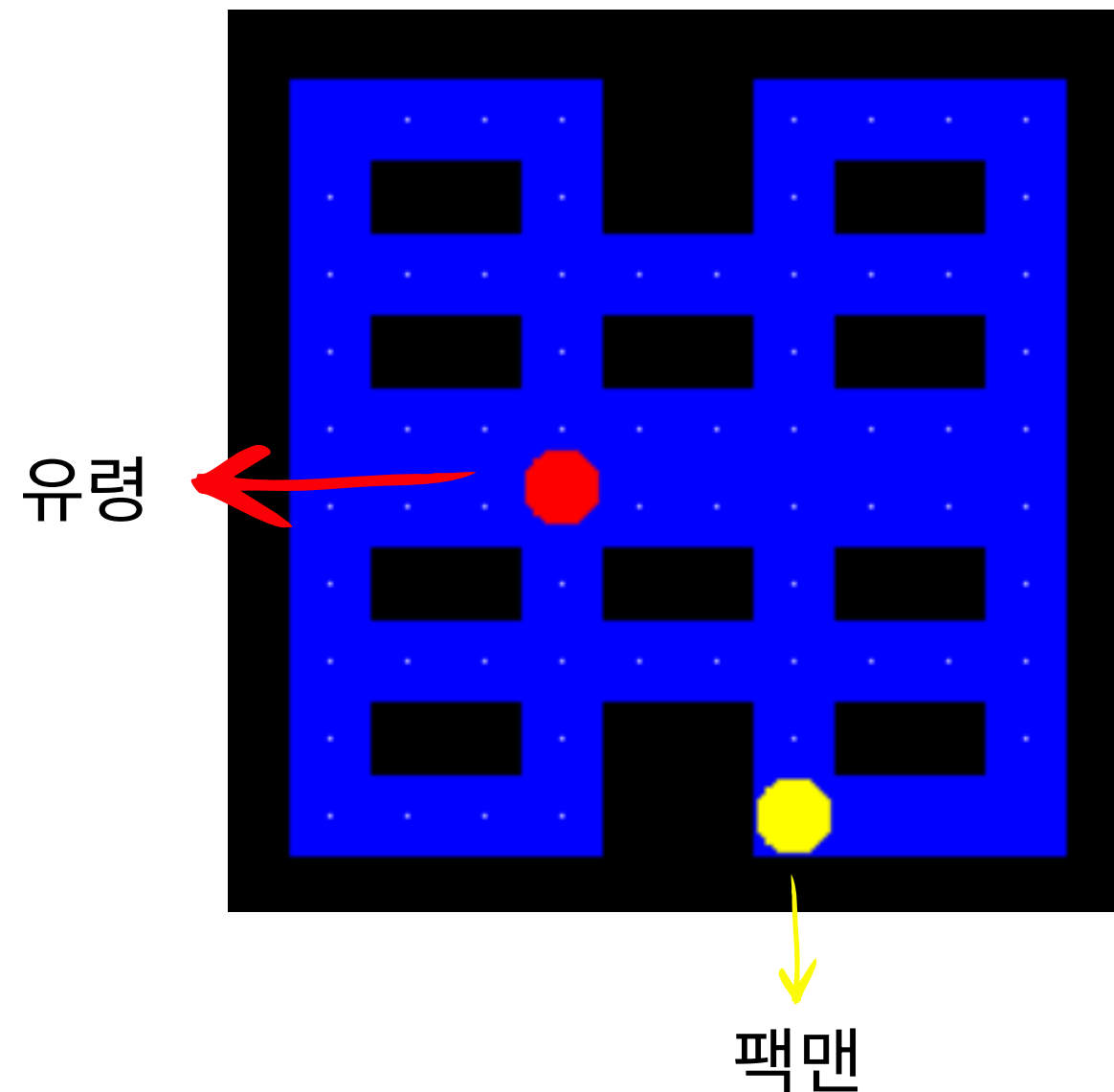
팩맨 환경에서 BFS 규칙 기반과 DQN 기반 에이전트의 성능 비교

민서진 팀 - 김민서, 임서진

목차

1. 게임 환경 설명
2. 규칙 기반 알고리즘
3. 학습 기반 알고리즘
4. DQN 환경 셋팅
5. 실험 환경
6. 실험 결과
7. DQN 결과 분석
8. 개선 방안
9. 한계

1. 게임 환경 설명 - 게임 목표, 맵 구성



- 게임 목표: 팩맨이 유령과 충돌하지 않으면서 제한된 step 내에 가능한 많은 먹이 수집하기
- 맵 구성
 - 12×12 크기의 고정된 타일 기반 미로
 - 유령 1마리
 - 벽과 이동 가능한 통로로 구성
 - 이동 가능한 통로에 먹이 배치
 - 먹이 1개 획득 시 게임 점수 1점
 - 모든 먹이를 획득하면 맵 클리어
 - 유령과 충돌 시 맵 종료

1. 게임 환경 설명 - 팩맨과 유령의 이동 알고리즘

- 팩맨

- 행동 공간: 상,하,좌,우
- 매 step마다 행동 하나를 선택
- 행동 한 번 당 한 타일 이동
- 별도의 정지 행동은 없음
- 먹이가 있는 타일에 도착하면 먹이 획득

- 유령: 방향 지속형 무작위 이동

현재 방향으로 이동할 수 있는가?

└ Yes → 같은 방향으로 한 타일 직진

└ No → 이동 가능한 방향 중 하나를 무작위 선택

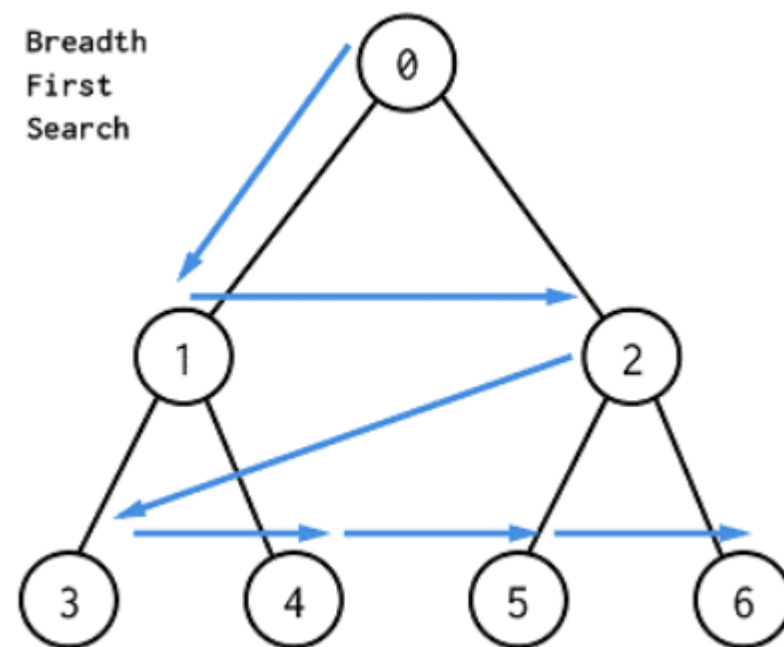
- 팩맨을 직접 추격하지 않음
- 앞이 벽일 때만 방향 변경
- 이동 가능한 방향을 균등한 확률로 선택
- 선택한 방향으로 같은 step에 한 타일 이동

1. 게임 환경 설명 - 한 step의 진행과 종료 조건

- 한 step의 진행 순서
 - 행동 선택
 - 팩맨 이동, 먹이 획득 처리, 유령 이동
 - 충돌, 종료 조건 검사
 - 다음 상태 반환
- 충돌 조건
 - 팩맨과 유령이 이동 후 같은 타일에 위치
 - 한 step 동안 서로의 이전 타일을 교환
- 에피소드 종료 조건
 - 유령과 충돌
 - 모든 먹이 획득
 - 팩맨의 이동 수 1,000 step 도달

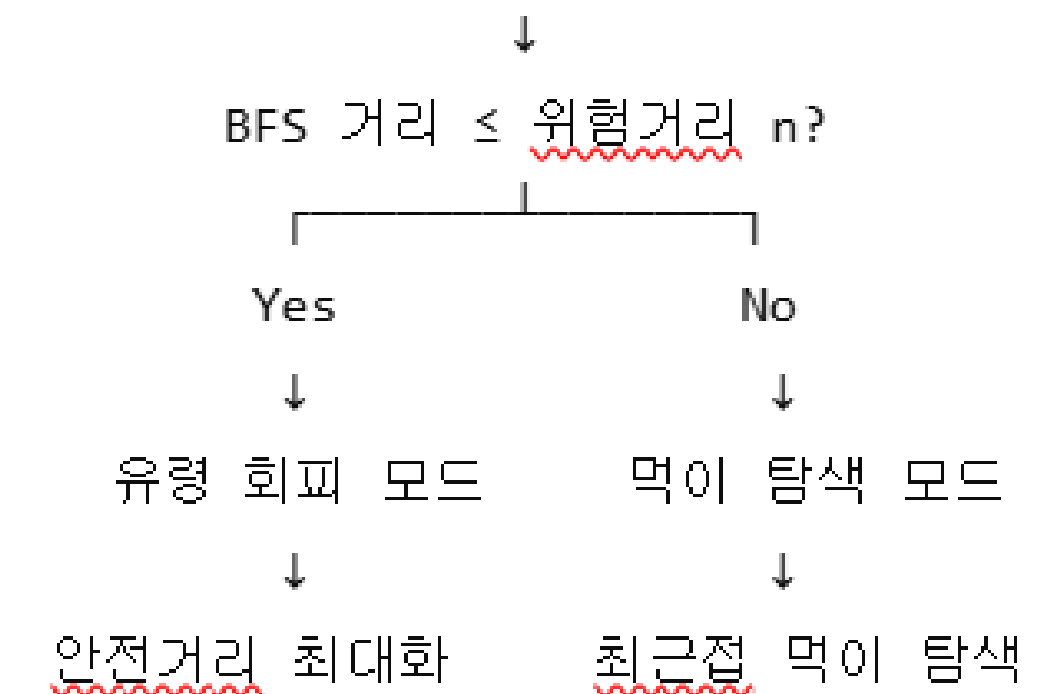
2. 규칙 기반 알고리즘 - 전체 구조

- Breadth-First Search (BFS, 너비 우선 탐색)
 - 최단 거리 알고리즘
 - 팩맨과 유령과의 거리, 팩맨으로부터 가장 가까운 먹이를 탐색할 때 사용

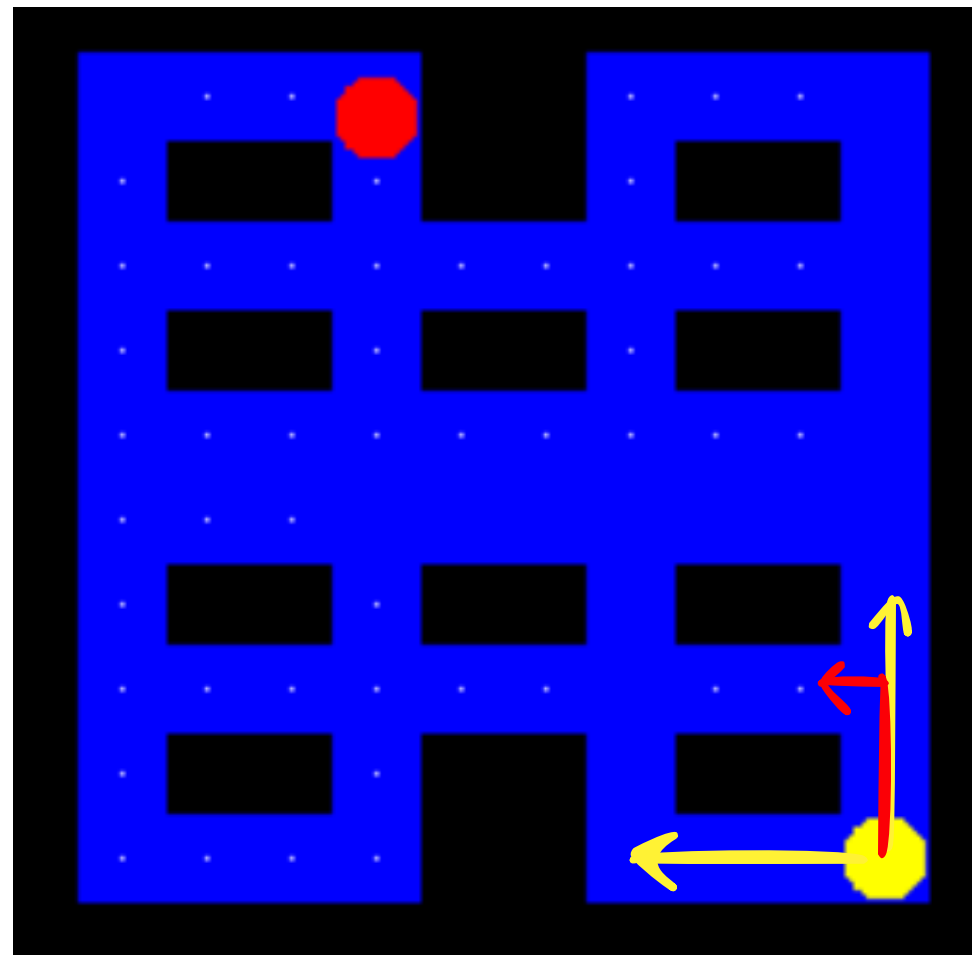


- 유령과의 거리에 따라 두 가지 모드로 전환

팩맨과 유령 사이의 BFS 거리 계산

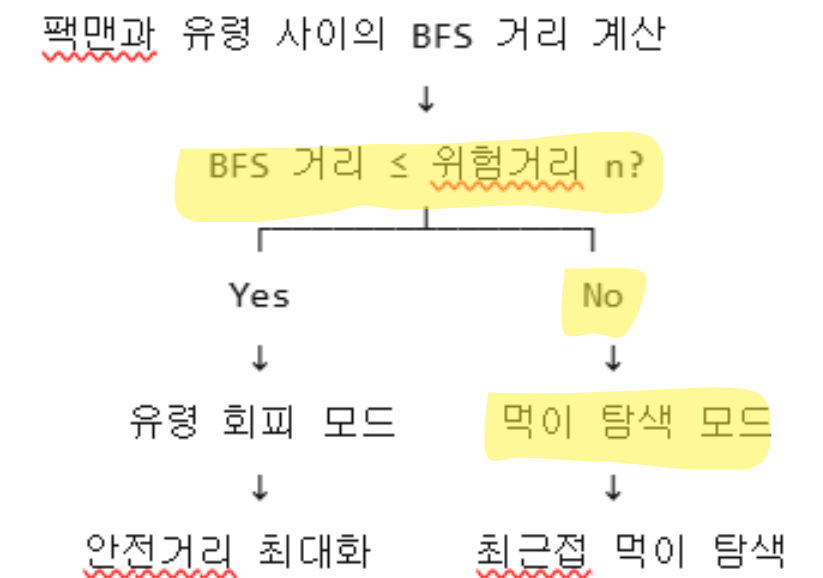


2. 규칙 기반 알고리즘 - 먹이 탐색 모드



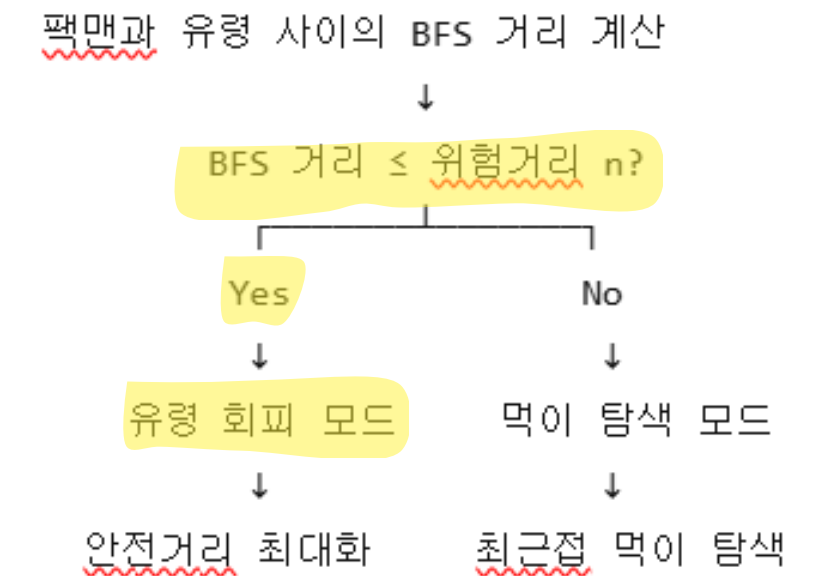
빨간색 경로가 먹이까지의 최단 경로

- BFS를 통해 가장 가까운 먹이를 탐색
 - 팩맨의 현재 위치에서 BFS 수행
 - 처음 발견한 먹이를 최근접 먹이로 선택
 - 최단경로의 첫 번째 행동만 실행
 - 다음 step에서 다시 경로 계산



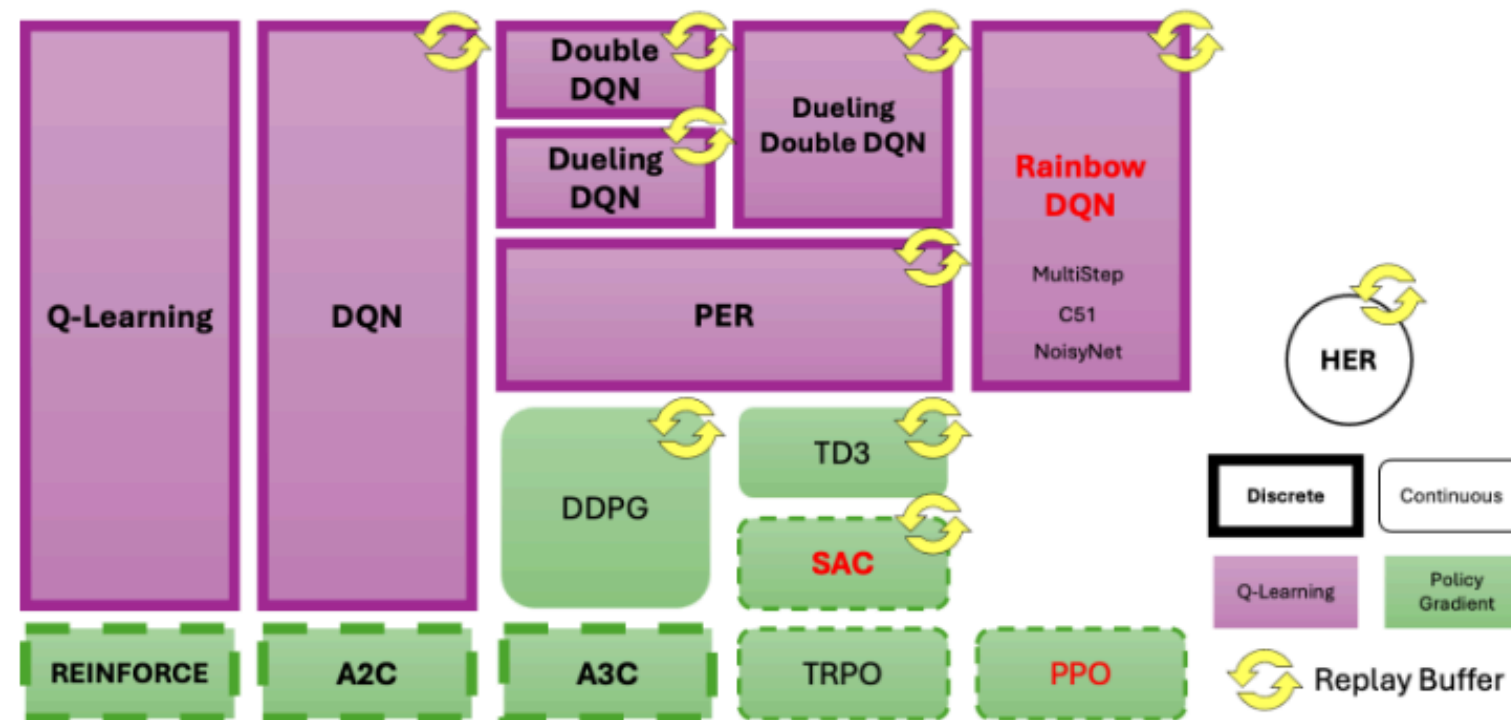
2. 규칙 기반 알고리즘 - 유령 회피 모드

- 유령의 가능한 다음 위치를 고려하여 가장 안전한 행동 선택
- Case 1. 유령이 직진할 수 있는 경우
 - 유령의 다음 위치를 하나로 예측 가능
 - 각 팩맨 후보 행동마다:
 - i. 팩맨의 이동 후 위치에서 유령의 예상 다음 위치까지 BFS 최단거리 계산
 - ii. 계산된 최단거리가 가장 큰 팩맨 행동 선택
- Case 2. 유령 앞이 벽인 경우
 - 유령의 다음 위치를 하나로 예측 불가능
 - 각 팩맨 후보 행동마다:
 - i. 팩맨의 이동 후 위치에서 가능한 모든 유령 다음 위치까지 BFS 최단거리 계산
 - ii. 그중 가장 짧은 거리를 유령이 가장 가까이 오는 **최악의 경우 거리**로 설정
 - iii. **최악의 경우 거리**가 가장 큰 팩맨 행동 선택



3. 학습 기반 알고리즘 - DQN

- DQN 선택 이유
 - discrete action 처리 가능
 - replay buffer를 통해 과거 경험 재사용
 - 알고리즘 해석이 상대적으로 간단



<강화학습 알고리즘>

- DQN 특징
 - Q-learning 확장 알고리즘
 - discrete action 처리
 - off-policy 방식
 - replay buffer 사용
 - ϵ -greedy를 통한 탐험과 활용의 균형
 - 이중 신경망 구조 (Q-network, Target network)

4. DQN 환경 셋팅 - Observation 설계

구성	300	159
벽 마스크	144	0
먹이 마스크	144	144
팩맨 위치	2	2
이동 가능 행동	4	4
유령 위치	2	2
유령 진행 방향 (one-hot)	4	4
유령 상대 dx, dy	-	2
남은 먹이 비율	-	1
합계	300	159

- 맵 고정 → 벽마스크 144 차원은 모든 관측에서 값이 동일, 정보량 0이므로 제거
- 먹이 마스크는 절대 좌표 유지
 - 자기중심 crop은 창 밖 먹이를 알 수 없어 전체 클리어가 어려움
 - 전역 BFS를 쓰는 규칙 기반과 비교도 불공정해짐
- 유령 진행 방향은 필수
 - 유령 이동의 83.4%가 직진 → 방향을 알면 다음 위치 예측 가능
 - 규칙 기반과 동일한 정보를 제공하되, 규칙 추출은 DQN의 몫

4. DQN 환경 셋팅 - 하이퍼파라미터 설계

파라미터	값	근거
net_arch	[128, 128]	입력이 300 → 159 축소
learning_rate	1e-4	부트스트래핑 구조상 큰 학습률은 발산 위험
batch_size	128	배치 확대 → 그래디언트 노이즈 감소, 학습 안정화
buffer_size	50,000	최근 10%의 경험 보관
learning_starts	3,000	버퍼가 다양해진 뒤 학습 시작
gamma	0.99	표준값
train_freq / gradient_steps	4 / 1	표준값
target_update_interval	1,000	
exploration	1.0 → 0.05, fraction 0.3	전체의 30% 지점에서 최종값 도달
max_grad_norm / tau	10 / 1.0	표준값 유지
학습 시드	42, 43, 44	신경망 초기 가중치. 유령 패턴과 무관

4. DQN 환경 셋팅 - 보상 설계

항목	값
먹이 획득	+1.0
매 step	-0.01
벽 방향 선택	-0.05
유령 충돌	-10.0 → -30.0 (2차에서 변경)
전체 클리어	+100.0

보상은 config.py 수정 없이 PacmanEnv(rewards=...)로 전달
규칙 기반 평가에는 영향 없음

- **1차 실험 충돌 -10**: 죽는 쪽이 이득인 구조
먹이 47개 수집 후 충돌 : 37
조심해서 30개만 수집: 30

손익분기 10개 < 실제 수집량 47개 → 패널티 무력화
에이전트는 이 구조를 학습해 회피를 포기

- **2차 -30으로 상향 했지만 충돌률 84.7% → 83.3%**
3배 상향에도 1.4% 감소에 그침
→ 회피 실패의 주원인은 보상 구조가 아님

5. 실험 환경 - 평가 seed 설계

- 조건 선택에 반복 사용한 대역에서 성능이 부풀려짐
- 최종 성능은 미사용 seed에서 측정해야 함
- 동일 seed 목록을 모든 에이전트에 적용 → 판별 대응 비교 가능

대역	용도
0 ~ 99	규칙 기반의 위험거리 n 선택 (개발용)
0 ~ 9,999	DQN 학습 중 유령 패턴
10000 ~ 10099	DQN 학습 조건 선택 (보상·학습량 비교)
30000 ~ 30099	최종 성능 평가 — 두 에이전트 공통, 100판

모델 (800만 step)	seed 10000번대	seed 30000번대	차이
시드 42	68.34	63.15	-5.19
시드 43	66.13	64.52	-1.61
시드 44	69.30	60.63	-8.67
평균	67.92	62.77	-5.15 (-7.6%)

5. 실험 환경 및 평가 지표

- 실험 환경

- 동일 조건

- 12×12 맵
 - 벽과 먹이 배치
 - 팩맨·유령 시작 위치
 - 유령 수와 이동 알고리즘
 - 상,하,좌,우 행동 공간
 - 충돌 및 종료 조건
 - 최대 길이: 1,000 step
 - 평가 seed와 평가 횟수

- 차이점

- 팩맨의 행동 결정 방법
 - 규칙기반 → BFS 알고리즘
 - 강화학습 → DQN 알고리즘

- 평가 지표

- 평균 게임 점수
 - 먹이 획득률
 - 평균 생존 step
 - 충돌률
 - 맵 클리어율

6. 실험 결과

- 규칙 기반 - BFS
 - 위험거리 $n = 3, 4, 5$ 비교 → 세 조건 모두 100% 클리어, $n=3$ 이 가장 효율적 (103 step)
 - 최종: 평균 70.00개 / 획득률 100% / 클리어율 100% / 충돌률 0%

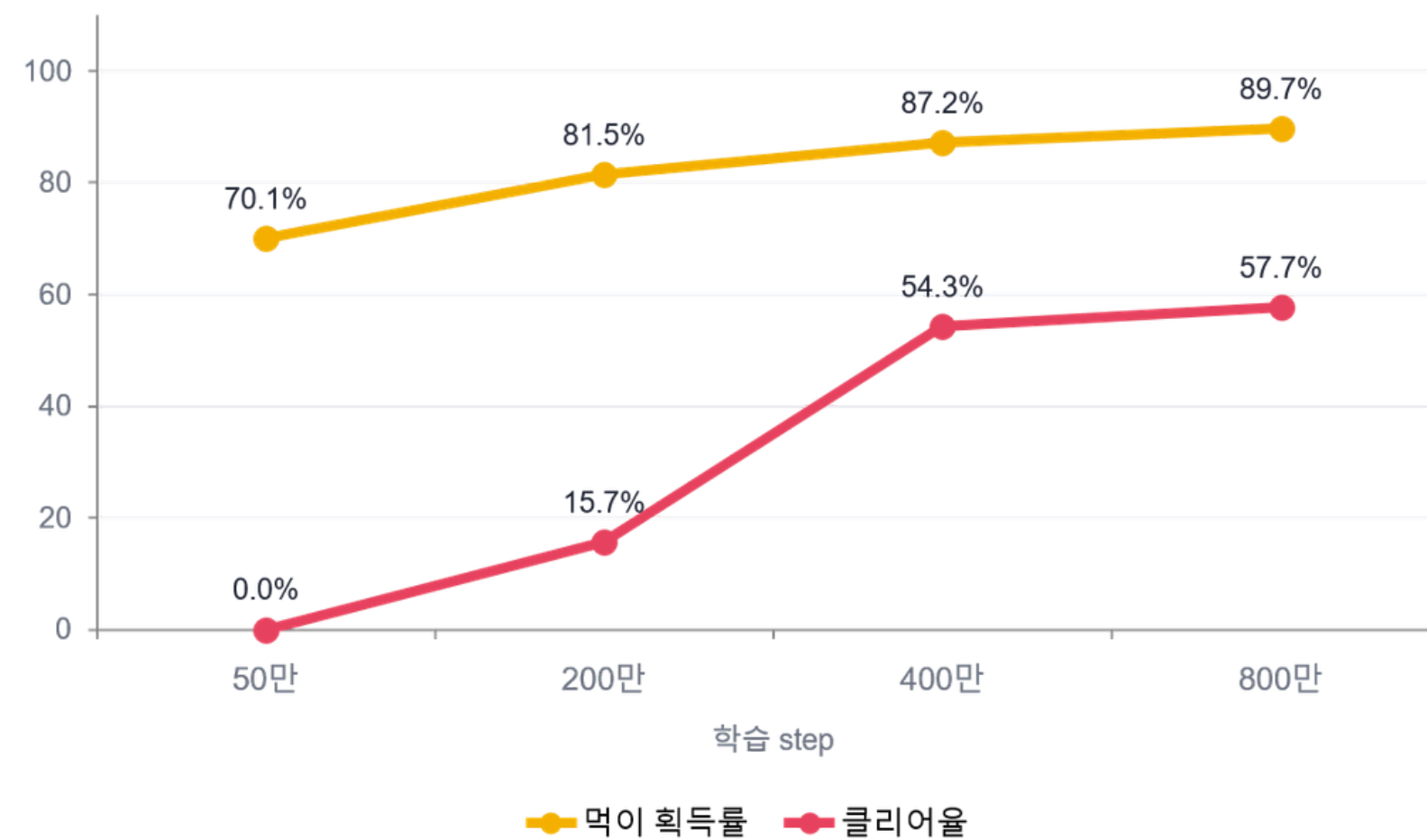
- 학습 기반 - DQN

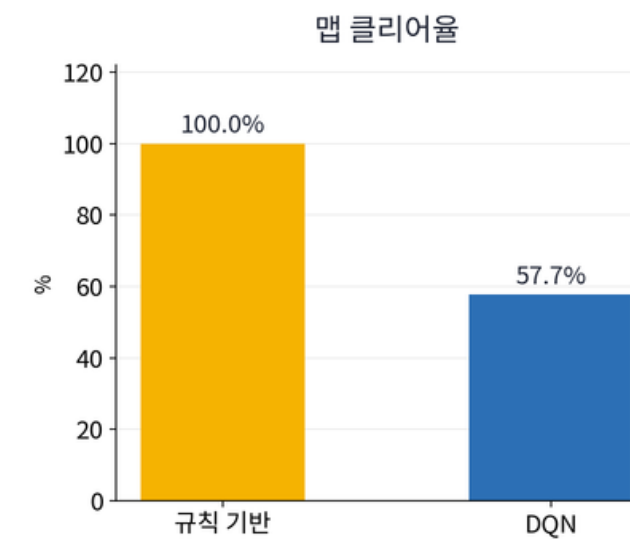
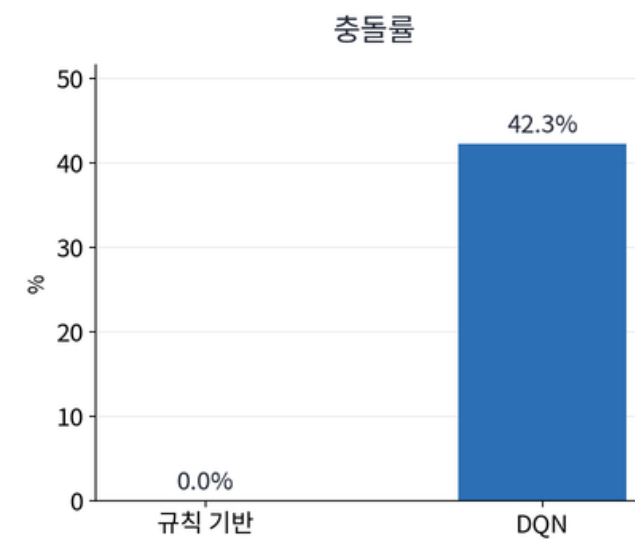
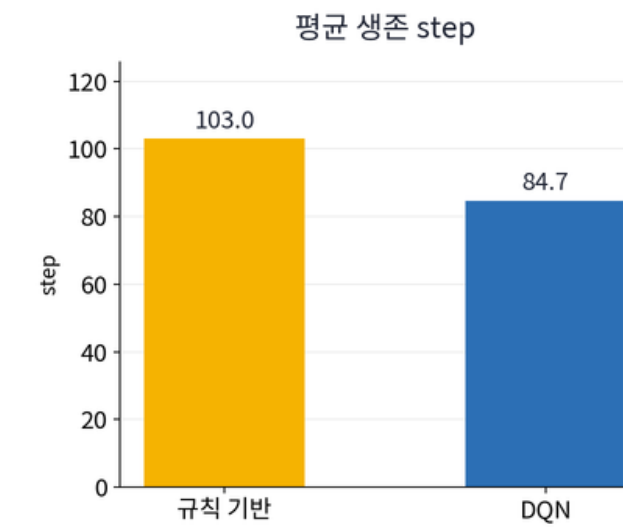
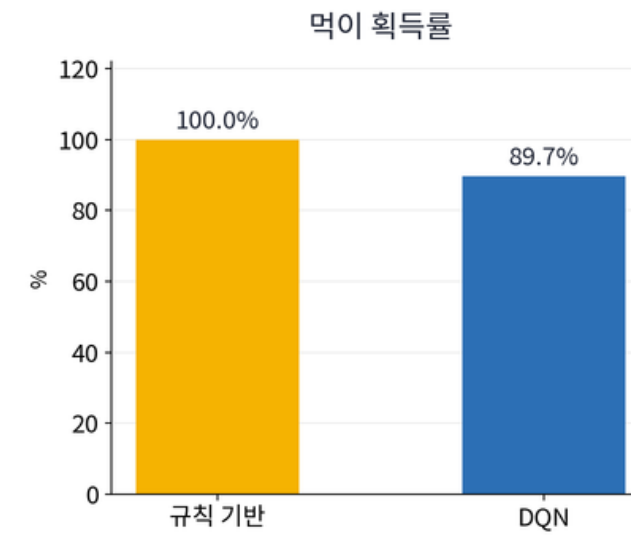
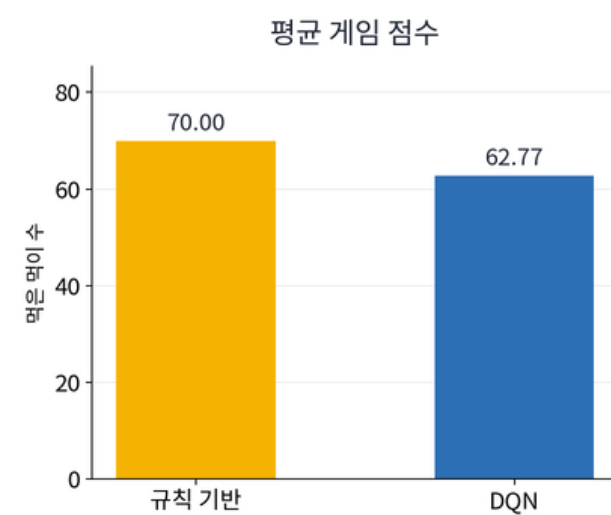
차수	변경점	평균 먹이	획득률	클리어율	충돌률	평균 step	시드 편차
1차	충돌 -10, 50만 step	47.21	67.4%	0%	84.7%	95.0	± 0.76
2차	충돌 -30, 50만 step	49.10	70.1%	0%	83.3%	107.9	± 5.34
3차	200만 step	57.05	81.5%	15.7%	69.7%	105.6	± 1.96
4차	400만 step	61.02	87.2%	54.3%	45.7%	80.2	± 8.70
5차	800만 step	62.77	89.7%	57.7%	42.3%	84.7	± 1.97

- 클리어는 200만 step에서 처음 발생 - 1~2차는 클리어 경험 0회
- 2차의 큰 편차는 수렴 부족

- 학습량 효과

- 개선폭 11.4 → 5.7 → 2.5%p - 학습량 2배마다 개선폭 절반씩 감소, 수확 체감 구간 진입
- 학습량만으로는 추가 향상에 한계





- 획득률 89.7%, 평균 62.77개 — 수집은 학습됨 (베이스라인 43%의 두 배 이상)
- 충돌률 42.3% — 회피를 일부 학습했으나 규칙 기반 0%와 격차
- 클리어율 57.7% + 충돌률 42.3% = 100% → 실패한 판은 전부 충돌사
- 생존 step 84.7 — 규칙 기반이 다 먹는 103보다 이르게 종료

7. DQN 결과 분석 - ① 회피 모듈

규칙 기반이 먹이 70개를 다 먹는 건 BFS때문인가 회피인가?

에이전트	BFS 먹이 탐색	유령 회피	평균 먹이
rule_n3	O	O	70.00개
greedy	O	X	30.66개
random	X	X	30.44개

- greedy $\approx$ random $\rightarrow$ BFS만으로는 랜덤 수준
- 70개의 근거는 회피 로직
- DQN 충돌률 42.3% $\rightarrow$ 회피를 절반만 학습

7. DQN 결과 분석 - ② 주어진 정보는 같은데 결과가 다른 이유

두 에이전트 모두 유령 위치와 진행 방향을 받는다

규칙 기반

- 유령 이동 규칙을 함수로 갖고 시작
 - 앞이 통로 → 직진 (83.4%) → 다음 칸 확정
 - 앞이 벽 → 방향 전환 (16.6%) → 모두 회피

→ 충돌률 0%

DQN

- 같은 정보를 받지만 규칙은 직접 알아내야 함
- 유령 위치 2차원 + 진행 방향 4차원
- 수천 판의 시행착오로 근사

→ 충돌률 42.3%

8. 개선 방안

- **먹이 수 축소** : 클리어율 57.7%의 병목 = 흩어진 마지막 먹이 탐색 구간
먹이 축소 → 병목 제거, 클리어 경험 증가로 +100 보상 작동
- **거리 기반 보상 세이핑** : 가장 가까운 먹이에 가까워지면 +0.1, 멀어지면 -0.1
현재는 먹어야만 보상 발생 → 신호가 희소
거리는 BFS가 아닌 직선거리 사용 (BFS는 규칙 기반의 정답을 주는 셈)

9. 한계

- 유령 1마리 → 규칙 기반 100% 클리어, 비교 폭이 좁음
- 맵 1종 고정 → 일반화 미검증

감사합니다